

EL ROL ESTRATÉGICO DEL MOVIMIENTO (UX/UI)

La animación en la web moderna no es un adorno, es una herramienta funcional. Su objetivo principal es:

- **Guiar la atención:** Dirigir la mirada del usuario hacia elementos clave como botones de compra o badges de oferta.
- **Proporcionar Feedback:** Confirmar que una acción se ha procesado (ej. un botón que "late" al pasar el cursor).
- **Reducir la carga cognitiva:** Las transiciones suaves entre estados evitan cambios bruscos que confunden al cerebro.
- **Comunicar estados del sistema:** Los *loaders* o indicadores de carga mantienen al usuario informado de que el proceso sigue activo, reduciendo la frustración.

ANIMACIONES NATIVAS CON CSS (EL MOTOR DEL MOVIMIENTO)

Para crear movimiento fluido usando solo CSS, contamos con dos herramientas principales dependiendo de la complejidad que busquemos: las Animaciones (Keyframes) y las Transiciones.

Animaciones Complejas: @keyframes y animation

Se utilizan cuando queremos que un elemento pase por múltiples estados, se mueva de forma autónoma o ejecute un bucle continuo. Se divide en dos partes:

- **@keyframes** (El Guion) Define la trayectoria y los estados por los que pasa un elemento a lo largo del tiempo, actuando como los fotogramas de una película.

Puntos de control: Se pueden usar las palabras `from` (0%) y `to` (100%) para cambios simples de un punto A a un punto B, o porcentajes intermedios (ej. 0%, 25%, 50%, 100%) para diseñar trayectorias complejas de múltiples pasos.

Propiedades de control (El Director) Para que el guion del `@keyframes` se ejecute, debemos aplicarlo al elemento usando las subpropiedades de `animation`:

- **animation-name** -> El nombre exacto del keyframe que queremos ejecutar.
- **animation-duration** -> El tiempo total que tarda en completarse una vuelta de la animación (ej. 2s, 500ms).
- **animation-timing-function** -> La curva de aceleración o ritmo. Valores comunes: ease (natural), linear (velocidad mecánica/constante), ease-in-out (arranque y frenada suaves), o cubic-bezier() para diseñar curvas 100% personalizadas.
- **animation-iteration-count** -> El número de veces que se repite la secuencia (puede ser un número exacto o infinite).
- **animation-direction** -> Define el sentido. El valor alternate es muy útil porque hace que la animación vaya y vuelva de forma fluida (como un péndulo), en lugar de reiniciarse de golpe al principio.
- **animation-fill-mode** -> Define cómo queda el elemento al terminar. El valor forwards obliga al elemento a quedarse congelado en el estado exacto de su último fotograma, evitando que regrese a su posición inicial.

Cambios Directos: Las Transiciones (transition)

A diferencia de los keyframes, las transiciones están diseñadas para interpolar suavemente el paso entre dos únicos estados cuando ocurre un evento concreto (por ejemplo, cuando el usuario pasa el ratón por encima con :hover o al añadir una clase con JavaScript).

• **Propiedades de control de transition**

- **transition-property** -> Indica a qué propiedad CSS específica se le aplicará el efecto (ej. background-color, transform, opacity, o all para animar todo lo que cambie).
- **transition-duration** -> El tiempo que tarda en completarse el cambio (ej. 0.3s).
- **transition-timing-function** -> La curva de aceleración del cambio, usando los mismos valores que en las animaciones (ease, linear, ease-in-out...).

- **transition-delay** -> Un tiempo de espera opcional antes de que el efecto comience a ejecutarse.

(Atajo práctico: Se suelen escribir todas juntas en una sola línea. Ej:
transition: background-color 0.3s ease-in-out;)

Resumen: ¿Cuándo usar cada una?

transition -> para microinteracciones simples, directas y reactivas (A -> B).

Ejemplo: Un botón que cambia de color suavemente al pasar el cursor.

animation -> para secuencias autónomas, complejas, multicapa o infinitas.

Ejemplo: Un icono de carga que gira continuamente o un elemento que rebota varias veces al entrar a la pantalla.

HERRAMIENTAS AVANZADAS (LIBRERÍAS)

Animate.css (Basado en CSS puro)

La Filosofía de Animate.css (CSS "Plug and Play") -> Su filosofía es ofrecer un catálogo enorme de animaciones @keyframes ya escritas y listas para usar. Ideal para añadir animaciones rápidas, predefinidas y eficientes.

- **Uso en el Proyecto:** Se importa mediante CDN en el head y se activa con la clase obligatoria `animate__animated`.
- **Estructura base:** Se debe añadir siempre la clase `animate__animated`. Al ponerla, le dices al navegador: "Prepara a este elemento porque le voy a aplicar una animación de Animate.css".
- **Clases de movimiento:**
 - **Familia de Entradas (Fades / Zoom):**
 - **`animate__fadeIn`:** El elemento aparece suavemente.
 - **`animate__fadeInDown`:** Entra cayendo suavemente desde arriba.

- **animate__zoomIn**: Aparece creciendo desde el centro.
- **Familia de Salidas:**
 - **animate__fadeOutUp**: Se desvanece mientras sube.
- **Familia de Atención (Atraen el ojo del usuario):**
 - **animate__heartBeat**: Simula un latido (se expande y contrae rápido). Lo tienes usado en tu botón "Ver catálogo".
 - **animate__tada**: Un pequeño rebote festivo.
 - **animate__shakeX**: Tiembla horizontalmente (ideal para indicar un error en un formulario).
- **Familia de Especiales:**
 - **animate__jackInTheBox**: Entra rotando y escalando, como un muñeco de caja sorpresa. Lo tienes en tus tarjetas de producto.
 - **animate__backInLeft**: Entra desde lejos (con escala reducida) y rebota hacia adelante.
- **Clases Modificadoras (Afinando el Comportamiento):** Al igual que en las propiedades nativas de animation, Animate.css ofrece clases de utilidad para ajustar el tiempo y las repeticiones sin tener que escribir CSS extra:

- **Retardos (Delays):**

- **animate__delay-1s**, **animate__delay-2s**, **animate__delay-3s**...
animate__delay-5s.

Uso real: En un grid de productos, se usan estas clases para que las tarjetas no aparezcan todas de golpe, sino una tras otra (la primera sin delay, la segunda con 1s, la tercera con 2s).

- **Velocidad (Duration):**

- Por defecto, duran 1 segundo.
 - **animate__slow** (dura 2s), **animate__slower** (3s).
 - **animate__fast** (0.8s), **animate__faster** (0.5s).

- **Repeticiones (Iteration):**

- `animate__infinite`: La animación no para nunca (como tu botón principal).
- `animate__repeat-2`, `animate__repeat-3`.

¿Cuándo usar Animate.css?: Es la herramienta perfecta cuando necesitas **velocidad de desarrollo**. (Landing pages, prototipos rápidos, mensajes de éxito/error que entran y salen, o animar un titular principal al cargar la página).

Cuándo NO usarla: Cuando necesitas sincronizar animaciones complejas, cuando quieres que una animación responda a la posición del scroll (ScrollTrigger), o cuando quieres encadenar múltiples movimientos (Timeline). En esos casos, GSAP es el rey.

GSAP - GreenSock (Basado en JavaScript)

Es el estándar profesional para animaciones de alto rendimiento e interactivas. A diferencia de CSS, GSAP permite que la animación responda dinámicamente al usuario.

- **Cómo añadir GSAP a tu proyecto?**
 - **Vía CDN (Más rápido)**: Pegando una etiqueta `<script>` en el head del docum. HTML.
 - **Vía NPM (Más profesional)**: Ejecutando `npm install gsap` en la terminal e importándolo (`import { gsap } from "gsap";`). Es el estándar si se usa React, Vue o empaquetadores como Vite.
- **¿Cómo se usa?**
 - **Identificación del elemento (El Selector)**
 - **`gsap.set(elemento, {...})`** -> Define el estado/coordenadas iniciales del elemento al momento de nacer.
 - **`gsap.to(elemento, {...})`** -> Define hacia dónde y cómo debe ir el elemento (estado final). En `gsap.to(elemento, {...})`, el "elemento" puede ser:
 - **Un selector de CSS**: Una cadena de texto como ".clase", "#id" o "button".

- **Un objeto del DOM:** Si ya tienes la referencia en una variable (por ejemplo, usando `document.getElementById`).
- **Ampliando los métodos principales:** Además de `set` y `to`, hay otros dos que se usan constantemente:
 - **`gsap.from(elemento, {...})`:** Define el estado inicial. La animación vendrá *desde* esos valores hasta como el elemento esté definido en tu CSS. Es perfecto para animaciones de entrada (ej: que algo aparezca desde abajo).
 - **`gsap.fromTo(elemento, {inicio}, {final})`:** Te da control total. Defines exactamente desde dónde empieza y exactamente dónde termina, ignorando los valores del CSS.
- **El objeto de configuración {...}**
 - **Propiedades de transformación: `x` e `y`** (para mover), `rotation`, `scale`, `skewX`. GSAP usa aceleración por hardware para estas.
 - **Propiedades CSS normales:** `opacity`, `backgroundColor`, `borderRadius`, etc.
 - **Propiedades especiales de GSAP:**
 - **`duration`:** Cuánto tarda (en segundos).
 - **`ease`:** La curva de velocidad (ej: `"power2.inOut"`, `"bounce.out"`, `"back"`). Es lo que da "personalidad" al movimiento.
 - **`delay`:** Cuánto tiempo esperar antes de empezar.
 - **`repeat`:** Cuántas veces repetir (-1 para infinito).
 - **`stagger`:** Si animas muchos elementos a la vez (ej: una lista), **`stagger: 0.1`** hará que cada uno empiece con 0.1s de diferencia respecto al anterior.
- **Timelines (Líneas de tiempo)** -> Aunque puede usarse `delay`, para animaciones complejas se usan las **Timelines**, que permiten encadenar animaciones sin calcular tiempos manualmente:

```
const tl = gsap.timeline();

tl.to(".caja", { x: 100, duration: 1 })

    .to(".caja", { y: 100, duration: 1 })

    .to(".caja", { rotation: 360 }); // Se ejecuta justo después de la anterior
```

- **Callbacks:** Los callbacks son funciones que se ejecutan en momentos clave:
 - **onStart:** Cuando la animación empieza.
 - **onUpdate:** En cada fotograma de la animación (útil para sincronizar otras cosas).
 - **onComplete:** Al terminar. Como mencionas, es vital para la limpieza de memoria: () => elemento.remove().

PATRONES DE DISEÑO UI/UX

Estructura Global y Variables CSS

- **:root** -> Permite declarar variables (ej. --bg: #0f1216;, --r: 18px;) para centralizar colores y medidas. Facilita los temas dinámicos y evita repetir "valores mágicos".
- **box-sizing: border-box** -> Fundamental para el diseño responsive; garantiza que el tamaño de las tarjetas no se desborde al añadirles *padding*. Hace que el *padding* y los bordes se incluyan dentro del ancho total, evitando que las cajas crezcan inesperadamente.
- **scroll-behavior: smooth** -> Crea un deslizamiento animado automático al hacer clic en enlaces internos (anclas de navegación).

Efectos Visuales

- **Glassmorphism (Efecto Cristal)** -> Se crea usando un fondo translúcido (rgba) sumado a backdrop-filter: blur(10px) para difuminar el contenido que pasa por detrás del contenedor.

- **Capas No Bloqueantes (pointer-events: none)** -> Esta propiedad es la "teoría de la transparencia táctil". Permite que el ratón "atraviese" el efecto visual para poder seguir haciendo clic en los botones que están debajo.
- **Marquee (Cinta en movimiento)** -> Se logra combinando contenedores con overflow: hidden, texto que no salte de línea (white-space: nowrap), y un @keyframes que desplaza el bloque infinitamente usando transform: translateX() en modo linear.

Posicionamiento Rápido

- **Centrar contenido:** display: grid; place-items: center; es el atajo más potente y moderno para centrar vertical y horizontalmente el contenido de una caja.
- **Superposición:** Usar position: relative en el padre y position: absolute en el hijo para anclar decoraciones (como emojis o brillos) en coordenadas exactas.

JAVASCRIPT, RENDIMIENTO Y OPTIMIZACIONES

Manejo del DOM

- **getElementById()** -> Selecciona exclusivamente por ID. Es más rápido en rendimiento.
- **querySelector()** -> Selecciona mediante cualquier selector CSS (clases, etiquetas). Es más versátil para búsquedas complejas.
- **Programación Orientada a Eventos:**
window.addEventListener("pointermove", (e) => {...}) reacciona a los movimientos del ratón/dedo, donde (e) captura todos los datos del evento (coordenadas clientX, etc.).

El Santo Grial de los 60 FPS

- **Priorizar transform y opacity:** Estas propiedades se procesan en la GPU (aceleración por hardware), evitando que el navegador tenga que recalcul todo el diseño (layout) de la página en cada frame.

- **Evitar propiedades de Layout:** Animar width, height, top o margin es costoso y provoca saltos visuales (jank).
- **Throttling en JavaScript:** Al animar según el movimiento del mouse, el script usa performance.now() para limitar la creación de partículas. Si el tiempo transcurrido es menor a 30ms, la función se detiene, ahorrando recursos masivos de CPU.
- **will-change:** Esta propiedad avisa al navegador qué elementos van a cambiar, permitiéndole optimizar el renderizado antes de que ocurra la acción.
- **La propiedad will-change:** Declarar will-change: transform, opacity; avisa al navegador que debe preparar un hilo de aceleración gráfica para ese elemento antes de que se mueva. Debe usarse sin abusar.